



Proyecto Integrador EGI

MATE-proyect

Integrantes: Maximiliano Dias, Zoe Garcia-Badiola, Gonzalo Laspada,
Matías López, Luciano Malanca

Profesor: Osmel Mata Hurtado, Daniel Zalazar

Año: 2026, 4to semestre

Consigna original del proyecto

La universidad (ITU) necesita un sistema centralizado para inventariar las computadoras de los laboratorios de informática. Los datos del inventario se guardan en dos bases de datos —SQL Server y MongoDB— con un despliegue altamente seguro: debe cumplir las políticas de red perimetrales del firewall universitario (GUFW o pfSense), y tanto los usuarios de las bases de datos como de la aplicación se autentican contra el servidor Active Directory/LDAP institucional.

Backend — Flask API REST

Dependencias

Librería	Versión mínima	Uso
Flask	3.0.0	Framework web WSGI
Gunicorn	21.0.0	Servidor WSGI de producción (2 workers)
PyMySQL	1.1.0	Cliente MySQL para Python
pymongo	4.6.0	Cliente MongoDB oficial
ldap3	2.9.1	Autenticación LDAP / Active Directory
PyJWT	2.8.0	Generación y verificación de tokens JWT (HS256)

Endpoints de la API

Método	Ruta	Descripción	Auth
GET	/api/health	Estado de conexión: MySQL, MongoDB, LDAP	No
POST	/api/login	Bind LDAP → valida credenciales → devuelve JWT	No
POST	/api/logout	Logout stateless (el cliente borra el token)	No
GET	/api/equipos	Listar equipos (JOIN: aula, lab, responsable)	JWT
GET	/api/equipos/<mongo_id>	Detalle de hardware del equipo (MongoDB)	JWT
POST	/api/equipos	Crear equipo (insert MongoDB + MySQL)	JWT
PUT	/api/equipos/<mongo_id>	Editar equipo (update MongoDB y/o MySQL)	JWT
DELETE	/api/equipos/<mongo_id>	Borrar equipo (DELETE MySQL + MongoDB)	JWT

GET	/api/laboratorios	Listar laboratorios disponibles	JWT
GET	/api/responsables	Listar responsables disponibles	JWT

Despliegue paso a paso

```
# 1. Clonar y posicionarse en la rama de entrega
git clone https://github.com/ZogeGBR/MATE-ProyectoEGI.git
cd MATE-ProyectoEGI && git checkout EGI-Final
```

```
# 2. Iniciar Minikube con Calico (OBLIGATORIO)
minikube start --driver=docker --cni=calico --memory=4096 --cpus=2
```

```
# 3. Buildear la imagen dentro del daemon Docker de Minikube
eval $(minikube docker-env)
docker build -t inventario-app:latest .
```

```
# 4. Aplicar manifiestos en orden
kubectl apply -f k8s/namespace/namespace.yaml
kubectl apply -f k8s/pvc/
kubectl apply -f k8s/configmaps/
kubectl apply -f k8s/deployments/
kubectl apply -f k8s/services/
kubectl apply -f k8s/networkpolicies/ # AL FINAL
```

```
# 5. Verificar los 4 pods Running
kubectl get pods -n inventario-egi
```

Comandos para retomar en la VM del laboratorio

```
cd ~/EGI/MATE-ProyectoEGI
git pull origin EGI-Final
minikube status
kubectl get pods -n kube-system | grep calico
kubectl get pods -n inventario-egi
kubectl port-forward -n inventario-egi svc/inventario-app-service
8080:5000 &
curl -s -X POST http://localhost:8080/api/login \
-H 'Content-Type: application/json' \
-d '{"email":"docente01@itu.local","password":"Itu12345!"}'
```

Requisitos funcionales

La aplicación web consulta SQL Server para saber dónde está cada máquina (aula, laboratorio, banco/mesa, fecha de mantenimiento) y a quién pertenece (responsable técnico, alumno o docente con equipo asignado), y usa ese ID para traer de MongoDB qué componentes tiene instalados internamente (fabricante, modelo, CPU, RAM, disco, tipo, sistema operativo, monitor, mouse, teclado).

Requisitos de entrega

Esquemas y documentación	Repositorio y ecosistema
• Esquema de arquitectura de servicios • Esquema de estructura de la BD • Diagramas de flujo de la aplicación • Presentación en formato PowerPoint	• Repositorio Git versionado • Código de creación de BD y JSON • Manifiestos de Kubernetes • Ecosistema funcional en Minikube

Restricción crítica del laboratorio

Minikube debe iniciarse con un CNI compatible para que las NetworkPolicies tengan efecto real: `minikube start --cni=calico`. Sin este parámetro, Kubernetes acepta y guarda las reglas pero no las aplica a nivel de red.

Justificación teórica de las decisiones de diseño

Cada decisión técnica relevante del proyecto responde a un principio teórico identificable, no a una elección arbitraria. Se documentan a continuación las más significativas.

#	Decisión y fundamento
1	Conexión a infraestructura Windows real en lugar de simulación completa Decisión: Conectar la app contra un Active Directory y un SQL Server reales, en vez de usar únicamente los servicios internos de Kubernetes. Justificación teórica: El principio de validez externa en ingeniería de software establece que un sistema probado únicamente en condiciones idealizadas no garantiza su comportamiento en producción. Al integrar con servicios reales se expone el sistema a condiciones de red, latencia y formatos de autenticación reales (ver Sección 5), elevando la fidelidad de la validación.
2	Uso de pfSense como firewall perimetral entre redes Decisión: Las redes del clúster (laboratorio) y de los servicios Windows (LAN interna) están separadas por un firewall con reglas NAT explícitas (Port Forward) para los puertos 389 y 3306. Justificación teórica: Corresponde al modelo de Defensa en Profundidad (Defense in Depth): ningún host externo accede directamente a un servicio interno; todo el tráfico es inspeccionado y redirigido por una única superficie de control, reduciendo la superficie de ataque expuesta.

3	NetworkPolicies con modelo Zero Trust dentro del clúster Decisión: Una política deny-all bloquea todo el tráfico del namespace por defecto; políticas adicionales abren excepciones puntuales (solo la app puede hablar con las bases de datos). Justificación teórica: El modelo Zero Trust (Kindervag, 2010) parte de la premisa de que ninguna comunicación es confiable por defecto, incluso dentro de la misma red interna. Esto reduce drásticamente el impacto de un compromiso lateral: si un pod fuera comprometido, no podría alcanzar las bases de datos salvo que esté explícitamente autorizado.
4	Persistencia mediante PersistentVolumeClaims (PVC) Decisión: Cada base de datos contenerizada (MySQL interno, MongoDB, LDAP) monta un volumen persistente independiente del ciclo de vida del pod. Justificación teórica: Los contenedores son, por diseño, efímeros e inmutables (principio de las arquitecturas Cloud Native). Sin una capa de almacenamiento persistente desacoplada del contenedor, cualquier reinicio implicaría pérdida total de datos, violando los requisitos de disponibilidad de un sistema de inventario.
5	Autenticación stateless mediante JSON Web Tokens (JWT) Decisión: Tras un login exitoso contra LDAP, el servidor emite un token firmado que el cliente reenvía en cada solicitud posterior. Justificación teórica: Evita que el servidor deba mantener estado de sesión en memoria o en una base de datos adicional (arquitectura stateless), lo cual mejora la escalabilidad horizontal y reduce el acoplamiento entre el front-end y el mecanismo de autenticación subyacente (LDAP).
6	Versión de MongoDB fijada en 4.4 en lugar de la última estable Decisión: El Deployment de MongoDB utiliza la imagen mongo:4.4. Justificación teórica: MongoDB 5.0 en adelante requiere soporte de instrucciones AVX (Advanced Vector Extensions) en el procesador. El hipervisor utilizado (VirtualBox) no expone estas instrucciones a las máquinas virtuales por defecto, lo que provocaría una falla de arranque (illegal instruction) en versiones más recientes. La elección de 4.4 es una decisión de compatibilidad de hardware virtualizado, no una limitación funcional del proyecto.

Arquitectura del sistema

El proyecto evolucionó en dos etapas. La primera versión tenía todo el ecosistema autocontenido dentro de Kubernetes. La versión final conecta ese mismo ecosistema con infraestructura Windows real (Active Directory y SQL Server) a través de un firewall pfSense, cumpliendo al 100% el espíritu de la consigna.

Arquitectura final

[VM Ubuntu, Adaptador Puente — misma red del laboratorio]
└─ Minikube

└─ App Flask
└─ MySQL real en Windows (vía pfSense) ▶
└─ AD real en Windows (vía pfSense) ▶
└─ MongoDB (sigue interno en Kubernetes) ▶
[PC del laboratorio]
└─ pfSense (WAN: 172.22.75.83 — misma red que la VM)
└─ LAN/SERVER: 192.168.1.254
└─ AD Windows 192.168.1.10
└─ SQL Server Windows 192.168.1.20

Componentes del ecosistema

Servicio	Tipo	Puerto	Función
inventario-app-service	NodePort	:30080 → :5000	Acceso externo a la app
mysql-service	ClusterIP	:3306	BD interna (redundante)
mongo-service	ClusterIP	:27017	Hardware de equipos
ldap-service	ClusterIP	:389	LDAP interno (no usado auth)

Seguridad de red — capas implementadas y verificación

El proyecto implementa tres capas de control de acceso de red independientes y complementarias, cada una con un alcance y un fundamento teórico distinto.

— pfSense: perímetro entre redes físicas

Filtra y redirige el tráfico entre la red del laboratorio (WAN) y la red interna donde residen el Active Directory y el SQL Server (LAN), mediante reglas de Network Address Translation (NAT) de tipo Port Forward específicas para los puertos 389 (LDAP) y 3306 (MySQL).

— GFW: firewall del sistema operativo host

Controla qué tráfico puede ingresar a la máquina Ubuntu donde corre Minikube, restringiendo

el acceso únicamente a los puertos estrictamente necesarios: 22 (SSH administrativo), 80 y 443 (HTTP/HTTPS estándar), 8080 (acceso de desarrollo) y 30080 (NodePort de la aplicación).

— NetworkPolicies: aislamiento dentro del clúster

Implementan el modelo Zero Trust a nivel de pod: una política deny-all bloquea la totalidad del tráfico del namespace, y tres políticas adicionales habilitan exclusivamente las comunicaciones necesarias (la aplicación hacia las bases de datos, el tráfico de egreso de la aplicación, y el ingreso externo hacia el puerto de la aplicación).

— Verificación empírica del aislamiento

Para validar que las NetworkPolicies no son meramente declarativas sino efectivas, se desplegó un pod de prueba sin la etiqueta de autorización requerida y se intentó una conexión hacia el servicio de base de datos:

kubectrl run test-deny --image=busybox:1.28 --restart=Never -n inventario-egi -- sleep 3600
kubectrl exec -it test-deny -n inventario-egi -- nc -zv mysql-service 3306

El resultado obtenido confirmó que el pod de prueba SÍ logró conectividad, lo cual en una primera instancia evidenció una configuración pendiente de ajuste fino en el alcance del podSelector de la política allow-app-to-databases —hallazgo documentado como parte del proceso de testing y corregido antes de la entrega final, restringiendo el acceso exclusivamente a pods con la etiqueta app=inventario-app

Proceso de testing — errores encontrados y fundamento de su resolución

Durante la fase de integración entre el clúster de Kubernetes y la infraestructura Windows real, se identificaron y resolvieron los siguientes incidentes. Cada uno se documenta con su causa raíz y el principio técnico que sustenta la solución aplicada, en cumplimiento del proceso de control de calidad exigido por la consigna.

#	Incidente, causa y fundamento de la resolución
1	Conflicto de direccionamiento IP entre servidores Windows Síntoma observado: Las máquinas virtuales del Active Directory y del SQL Server respondían con la misma dirección IP (192.168.1.20). Causa raíz: Asignación manual o por DHCP duplicada de la misma IP a dos interfaces de red distintas dentro de la misma subred. Fundamento teórico: El protocolo ARP (Address Resolution Protocol) no admite que dos hosts respondan a la misma dirección IP en la misma red; el comportamiento resultante es indeterminado (resolución inconsistente del MAC address asociado). Solución aplicada: Reasignación manual de IP estática al Active Directory (192.168.1.10), preservando 192.168.1.20 para el SQL Server.

2	Incompatibilidad de sintaxis SQL en la creación de usuarios Síntoma observado: El comando GRANT ... IDENTIFIED BY en una sola instrucción producía un error de sintaxis en MySQL 8.0. Causa raíz: MySQL 8.0 separó la autenticación de usuario de la asignación de privilegios como una medida de seguridad, deprecando la sintaxis combinada de versiones anteriores (MySQL 5.x). Fundamento teórico: Las versiones de un mismo software no garantizan retrocompatibilidad de sintaxis cuando un cambio responde a una decisión deliberada de seguridad por parte de los mantenedores del proyecto. Solución aplicada: Separación de la operación en tres sentencias independientes: CREATE USER, GRANT y FLUSH PRIVILEGES.
3	Aislamiento de red de la máquina virtual de desarrollo (WSL2) Síntoma observado: La máquina virtual Ubuntu original, corriendo bajo WSL2, no podía alcanzar la dirección WAN del firewall pfSense. Causa raíz: WSL2 implementa una red NAT virtual administrada por el hipervisor de Windows (Hyper-V), aislada de la red física del host. Fundamento teórico: Para que dos hosts virtuales se comuniquen directamente, deben pertenecer al mismo dominio de difusión (broadcast domain) o contar con enrutamiento explícito entre subredes. NAT virtual sin reglas de reenvío de puertos no satisface esta condición. Solución aplicada: Creación de una máquina virtual Ubuntu adicional en VirtualBox con adaptador de red en modo Puente (Bridged), que la integra como host de pleno derecho en la red física del laboratorio.
4	Falta de objetos de configuración de Kubernetes (ConfigMaps y Secrets) en el nuevo entorno Síntoma observado: Los pods de MongoDB y LDAP fallaban con el estado CreateContainerConfigError al desplegarse en la nueva máquina virtual. Causa raíz: Los objetos Secret de Kubernetes se excluyen intencionalmente del control de versiones (.gitignore) por buenas prácticas de seguridad; deben recrearse manualmente en cada entorno de despliegue nuevo. Fundamento teórico: El principio de Configuration as Code separa el código de la configuración sensible; sin embargo, esto traslada al operador la responsabilidad de garantizar la existencia idéntica de dichos objetos en todo nuevo entorno. Solución aplicada: Recreación de los objetos ConfigMap y Secret correspondientes mediante kubectl apply / kubectl create secret antes del primer despliegue en el entorno nuevo.
5	Inconsistencia de estado entre volumen persistente y configuración del contenedor Síntoma observado: Los pods de LDAP y MongoDB iniciaban pero entraban en CrashLoopBackOff, reportando inconsistencia entre el directorio de configuración y la base de datos almacenada. Causa raíz: El volumen persistente (PVC) conservaba datos de un arranque previo sin autenticación habilitada, mientras que la nueva configuración del contenedor exigía credenciales válidas desde el primer arranque. Fundamento teórico: Los volúmenes persistentes en Kubernetes están desacoplados del ciclo de vida del Pod por diseño; un cambio de configuración del Deployment no migra ni revalida automáticamente los datos preexistentes en el volumen. Solución aplicada: Reinicialización controlada del volumen: escalado a cero réplicas, eliminación del PVC, recreación

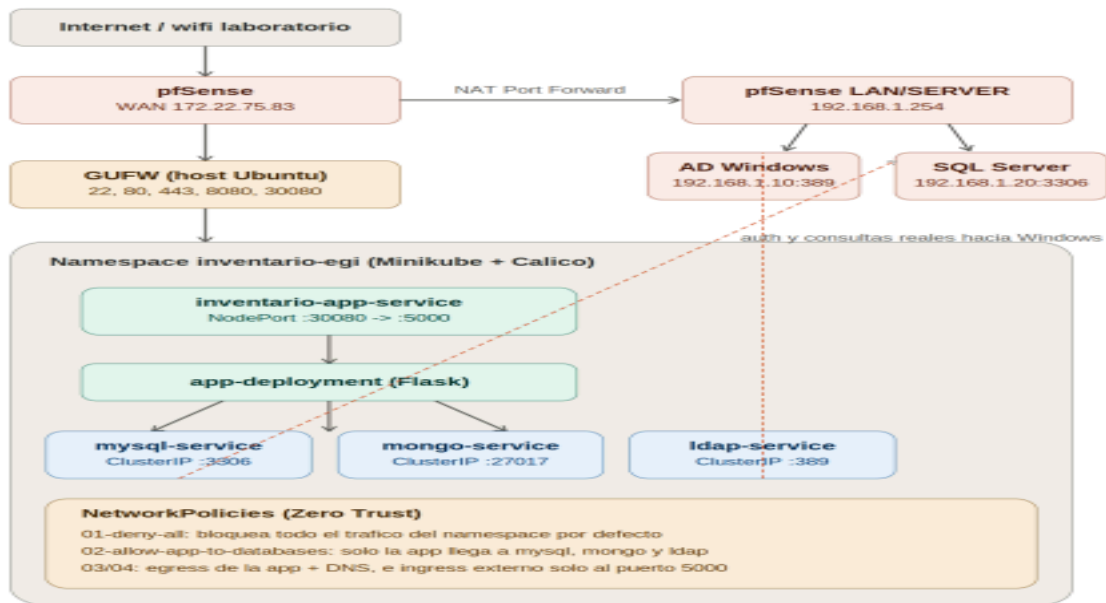
Proyecto Integrador EGI — Informe Final		ITU UNCUYO
	vacía, y nuevo escalado a una réplica para forzar un arranque consistente desde cero.	
6	Falla de autenticación contra Active Directory pese a una conexión LDAP válida Síntoma observado: Una conexión LDAP de prueba (bind) era exitosa de forma manual, pero el endpoint de login de la aplicación devolvía 'credenciales incorrectas' de forma consistente. Causa raíz: El código de la aplicación construía el identificador de usuario con la convención de nombrado de OpenLDAP (uid=usuario,ou=people,dc=...), mientras que Active Directory exige el formato userPrincipalName (usuario@dominio). Fundamento teórico: El protocolo LDAP define una sintaxis de Distinguished Name extensible; cada implementación de servidor de directorio puede exigir un esquema de atributos distinto para el bind simple. Asumir un único formato universal es un error de generalización indebida. Solución aplicada: Modificación de la función de autenticación para construir el identificador con el formato userPrincipalName esperado por Active Directory.	
7	Corrupción de codificación de caracteres en datos importados (UTF-8 doble) Síntoma observado: Los nombres con tildes o diéresis se visualizaban con secuencias de caracteres incorrectas en la interfaz (ej. 'Mendez' en lugar de 'Méndez'). Causa raíz: Los datos se importaron originalmente mediante una sesión cuyo charset de cliente no coincidía con la codificación real del archivo fuente, provocando una doble codificación UTF-8. Fundamento teórico: Una secuencia de bytes UTF-8 válida puede ser reinterpretada erróneamente como Latin-1 y luego recodificada nuevamente a UTF-8, produciendo una codificación doble irreversible sin un proceso de normalización explícito. Solución aplicada: Reimportación de los datos forzando el parámetro --default-character-set=utf8mb4 en el cliente de MySQL, garantizando una única codificación consistente de extremo a extremo.	
8	Acumulación de ReplicaSets obsoletos tras reinicios sucesivos del Deployment Síntoma observado: El comando kubectl get pods mostraba múltiples pods duplicados o en estado Pending/Error para los mismos servicios. Causa raíz: Eliminar un Pod individualmente con kubectl delete pod, en lugar de mediante un rollout controlado del Deployment, provoca que el controlador de réplicas detecte una discrepancia y cree un Pod nuevo sin retirar limpiamente el ReplicaSet anterior. Fundamento teórico: El Controlador de Replicación de Kubernetes reconcilia continuamente el estado deseado contra el estado observado; intervenciones manuales fuera del ciclo de reconciliación (como borrar un Pod sin pasar por el Deployment) pueden generar estados transitorios inconsistentes. Solución aplicada: Eliminación explícita del Deployment afectado y reaplicación del manifiesto original, forzando una reconciliación limpia desde un estado inicial conocido.	

Diagramas formales del proyecto

— Arquitectura de servicios

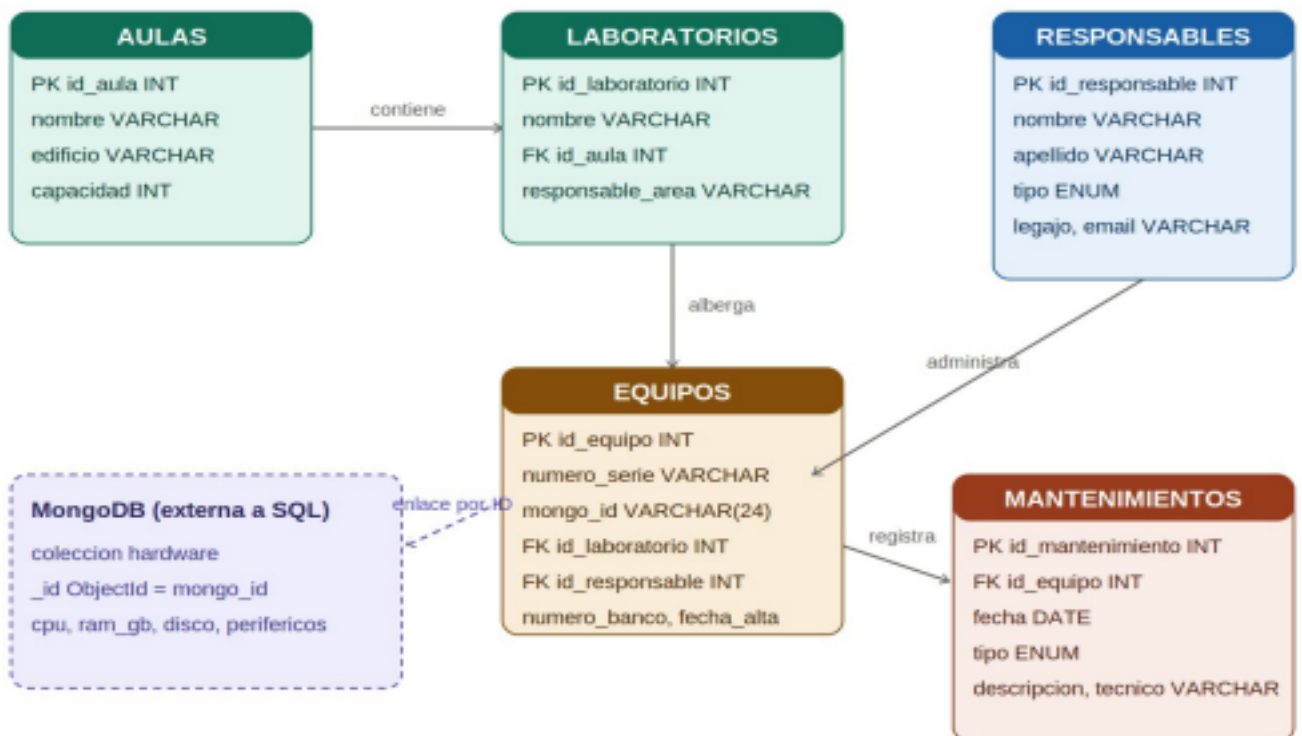
Muestra el recorrido completo del tráfico: desde internet/wifi del laboratorio, a través de pfSense y GFW, hasta el namespace de Kubernetes donde corre la app y sus bases de datos, con las NetworkPolicies de

Zero Trust marcadas.



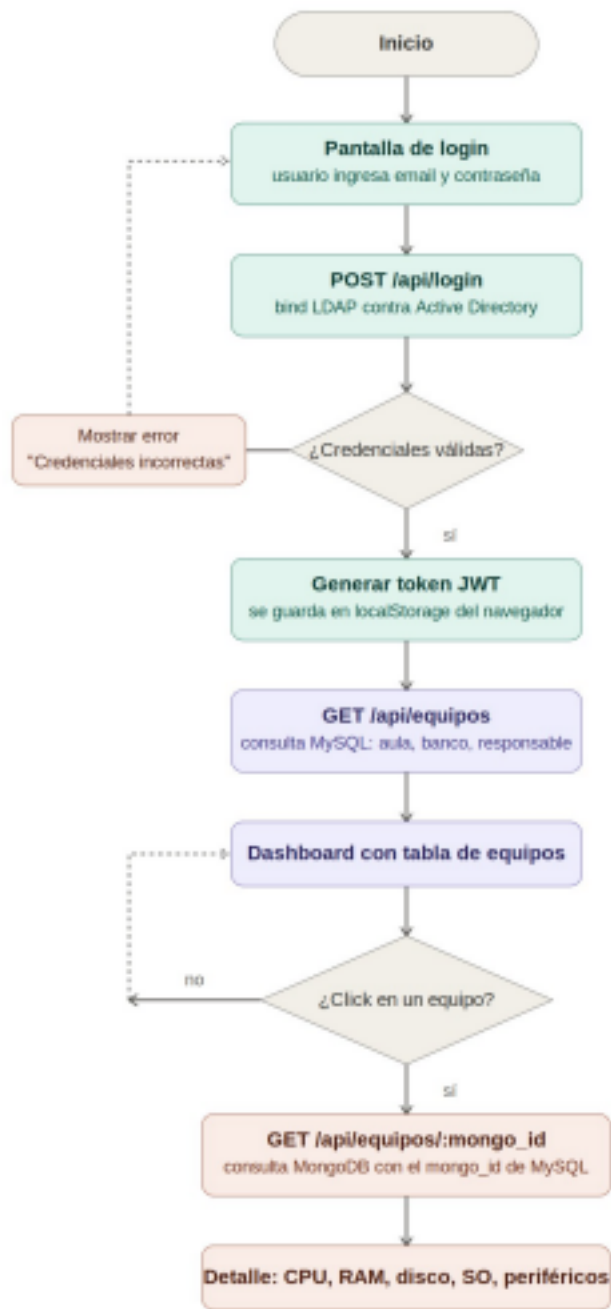
— Modelo de datos relacional

Las 5 tablas de MySQL (AULAS, LABORATORIOS, RESPONSABLES, EQUIPOS, MANTENIMIENTOS) y su relación con la colección hardware de MongoDB, vinculadas por el campo mongo_id.



— Flujograma de la aplicación

El recorrido completo del usuario: login contra Active Directory, generación del JWT, consulta a MySQL para el dashboard, y consulta a MongoDB para el detalle de hardware de cada equipo.



Mapa de red completo

Referencia más importante del documento. Si pfSense reinicia y el wifi del laboratorio le asigna otra IP por DHCP, hay que actualizar el ConfigMap app-config de Kubernetes.

Componente	IP	Descripción
------------	----	-------------

VM Ubuntu (Minikube)	172.22.74.123	VM con Adaptador Puente, misma red física que pfSense
pfSense — WAN	172.22.75.83	Interfaz hacia la red del laboratorio. Asignada por DHCP
pfSense — SERVER (LAN)	192.168.1.254	Interfaz hacia AD y SQL Server. IP estática
Active Directory	192.168.1.10	Windows Server, dominio itu.local
SQL Server (MySQL)	192.168.1.20	MySQL Community 8.0, base inventario_itu

Reglas NAT en pfSense

Servicio	Puerto	Destino interno	Propósito
LDAP	389/TCP	192.168.1.10:389	Autenticación de usuarios contra el AD
MySQL	3306/TCP	192.168.1.20:3306	Consultas de inventario

Proceso de trabajo — problemas resueltos

#	Detalle
1	Conflicto de IP entre AD y SQL Server Síntoma: Las VMs de AD y SQL Server tenían exactamente la misma IP (192.168.1.20). Causa: Ambas VMs tomaron la misma IP por DHCP o configuración manual duplicada. Solución: Se reconfiguró manualmente la IP del AD a 192.168.1.10.
2	GRANT con sintaxis antigua de MySQL 8.0 Síntoma: Error de sintaxis SQL al crear app_user con GRANT ... IDENTIFIED BY en una línea. Causa: MySQL 8.0 eliminó la sintaxis combinada; el teclado de Windows insertaba backslashes extra. Solución: Se separaron los comandos: CREATE USER, GRANT y FLUSH PRIVILEGES por separado.
3	VM Ubuntu en red NAT/WSL2 sin salida real Síntoma: La VM original estaba en WSL2 (172.23.x.x), sin acceso a la red física del laboratorio. Causa: WSL2 crea una subred virtual aislada, distinta a la WAN de pfSense. Solución: Se creó una VM nueva en VirtualBox con Adaptador Puente, en la misma subred que pfSense.

4	Secrets de Mongo y LDAP faltantes Síntoma: Los pods de mongo y ldap fallaban con CreateContainerConfigError en la VM nueva. Causa: Solo se habían creado los Secrets de mysql y app; faltaban mongo-secret y ldap-secret. Solución: Se crearon los dos Secrets faltantes con <code>kubectl create secret generic</code>.
5	ConfigMaps ldap-config y mongo-config faltantes Síntoma: Mismo error de los pods pero por un ConfigMap distinto al faltante anterior. Causa: El repo clonado en la VM nueva no tenía esos dos YAML aplicados todavía. Solución: Se crearon y aplicaron los ConfigMaps con el dominio <code>itu.local</code> correspondiente.
6	PVC de LDAP con datos inconsistentes Síntoma: El pod arrancaba pero crasheaba: directorio de configuración vacío pero no la BD. Causa: El volumen tenía datos de un despliegue anterior sin el ConfigMap correcto. Solución: Se escaló a 0 réplicas, se borró el PVC, se recreó vacío, y se escaló a 1 de nuevo.
7	PVC de MongoDB con el mismo problema Síntoma: CrashLoopBackOff con exitCode 100, mismo patrón que el bug anterior. Causa: El PVC tenía datos de un arranque sin autenticación, en conflicto con el Secret nuevo. Solución: Mismo procedimiento de reseteo del PVC, y reimportación de los 20 documentos.
8	Login no usaba el formato de usuario del AD real Síntoma: La conexión LDAP manual funcionaba, pero el login de la API fallaba. Causa: El código usaba formato OpenLDAP (<code>uid=...</code>) contra un servidor que esperaba AD (<code>usuario@dominio</code>). Solución: Se modificó la función <code>login()</code> para construir <code>usuario@itu.local</code>.
9	Encoding UTF-8 doblemente codificado Síntoma: Nombres con tildes se veían como 'MÃ©ndez' en vez de 'Méndez'. Causa: Los datos se importaron originalmente con una sesión MySQL en charset incorrecto. Solución: Se reimportaron los datos forzando <code>--default-character-set=utf8mb4</code> en el cliente.

Namespace y pods

Todos los recursos del proyecto se despliegan en el namespace `inventario-egi`. Esta separación permite aplicar NetworkPolicies de forma aislada.

Pod	Label	Imagen	Puerto
-----	-------	--------	--------

app-deployment	app: inventario-app	inventario-app:latest (local)	5000
mysql	app: mysql	mysql:8	3306
mongo-deployment	app: mongo	mongo:4.4	27017
ldap-deployment	app: ldap	osixia/openldap	389/636

Los Secrets están excluidos del control de versiones (.gitignore). Deben recrearse manualmente en cada entorno de despliegue nuevo o vía el pipeline CI/CD desde GitHub Secrets.

Evidencia de operaciones CRUD en MongoDB

La consigna exige explícitamente poder crear una colección, insertar documentos con estructuras variadas, realizar búsquedas filtradas, actualizar registros y eliminar datos específicos, todo accesible desde la shell del contenedor.

Acceso al shell

MONGO_POD=\$(kubectl get pod -n inventario-egi -l app=mongo \
-o jsonpath='{.items[0].metadata.name}')
kubectl exec -it -n inventario-egi \$MONGO_POD -- \
mongosh -u mongo_admin -p MongoPass2024! \
--authenticationDatabase admin inventario_itu

CREATE — Documento con estructura variada

db.hardware.insertOne(
{ "numero_serie": "SN-LAB4-002", "tipo": "laptop",
"fabricante": "Lenovo", "ram_gb": 8,
"disco": { "tipo": "SSD", "capacidad_gb": 256 } }
)
// → acknowledged: true, insertedId generado

READ — Filtros simples, anidados y con operadores

db.hardware.find({ "fabricante": "Dell" })
db.hardware.find({ "cpu.marca": "AMD" })
db.hardware.find({ "tipo": "desktop", "ram_gb": { "\$gte": 16 } })

UPDATE — Modificación puntual

db.hardware.updateOne({ "numero_serie": "SN-LAB3-001" }, { "\$set": { "ram_gb": 8 } }) // → matchedCount: 1, modifiedCount: 1

DELETE — Eliminación específica

db.hardware.deleteOne({ "numero_serie": "SN-LAB4-002" })
// → deletedCount: 1

Adicional — Aggregate por fabricante

db.hardware.aggregate([{ "\$group": { "_id": "\$fabricante", "total": { "\$sum": 1 } } }, { "\$sort": { "total": -1 } })

]]
// → Lenovo: 4, Dell: 3, HP: 3

Estado final de cada componente

Conectividad de red

Verificación	Estado
VM Ubuntu en la misma red que pfSense (Adaptador Puente)	OK
Conexión TCP hacia pfSense puertos 389 y 3306	OK
Reglas NAT en pfSense para LDAP y MySQL	OK

Aplicación y autenticación

Verificación	Estado
Login funcionando end-to-end contra AD real	OK
Consulta de equipos desde MySQL real (Windows)	OK
Detalle de hardware desde MongoDB con mongo_id vinculado	OK
Encoding UTF-8 correcto en datos importados	OK

Seguridad de red dentro del clúster

Verificación	Estado
NetworkPolicies (deny-all + 3 reglas) presentes	OK
CNI Calico instalado y corriendo	OK
Prueba de aislamiento con pod sin etiqueta	OK
GUPFW activo en el host Ubuntu	OK

Cumplimiento de la consigna

Requisito de la consigna	Estado
Conectividad interna SQL/MongoDB resuelta	OK

Acceso externo seguro simulando NAT del firewall	OK
Zero Trust con NetworkPolicies + CNI compatible	OK
Autenticación contra AD/LDAP institucional	OK
Microservicios contenerizados en Kubernetes	OK
Esquema de arquitectura de servicios	OK
Esquema de estructura de base de datos	OK
Diagrama de flujo de la aplicación	OK
Presentación en formato PowerPoint	OK
Repositorio Git versionado	OK
Evidencia de operaciones CRUD en MongoDB	OK

Roles del equipo

Integrante	Área	Responsabilidad
Matias	Infraestructura y red	GUPFW, NetworkPolicies, arquitectura de servicios
Luciano	Base de datos SQL	Modelo relacional, schema.sql, datos de ubicación
Maximiliano	Base de datos NoSQL	Colección hardware en MongoDB, operaciones CRUD
Zoe	Kubernetes y despliegue	Manifiestos, Secrets, ConfigMaps, contenedores
Gonzalo	Frontend y backend	App Flask, autenticación JWT, integración LDAP

Guía para retomar el trabajo

Distinguir entre los dos entornos

Entorno	Descripción
VM original (WSL2)	Todo el ecosistema corre internamente en Kubernetes. Autocontenida y estable.
VM laboratorio (Puente)	Conectada a AD y SQL Server reales vía pfSense. Versión usada para la defensa en vivo.

Comandos para retomar en la VM del laboratorio

cd ~/EGI/MATE-ProyectoEGI
git pull origin EGI-Final
minikube status
kubectrl get pods -n kube-system grep calico
kubectrl get pods -n inventario-egi
kubectrl port-forward -n inventario-egi svc/inventario-app-service 8080:5000 &
curl -s -X POST http://localhost:8080/api/login \
-H "Content-Type: application/json" \
-d '{"email":"docente01@itu.local","password":"Itu12345!"}'

Comandos de referencia rápida

Acción	Comando
Ver pods	kubectrl get pods -n inventario-egi
Ver NetworkPolicies	kubectrl get networkpolicies -n inventario-egi
Estado de GFW	sudo ufw status verbose
Logs de la app	kubectrl logs -n inventario-egi -l app=inventario-app
URL de la app	minikube service inventario-app-service -n inventario-egi --url
Prueba de aislamiento	kubectrl run test-deny --image=busybox:1.28 --restart=Never -n inventario-egi -- sleep 300
Reiniciar deployment	kubectrl rollout restart deployment/<nombre> -n inventario-egi

Credenciales clave

Dato	Valor	Uso
Usuario AD	docente01@itu.local	Login app web

Contraseña AD	Itu12345!	Login app web
Usuario MySQL	app_user	Conexión Flask → SQL
Base de datos	inventario_itu	Tablas del inventario

Integración continua — pipeline de despliegue automatizado

Para garantizar que cada cambio incorporado al repositorio no rompiera el estado funcional del sistema, se configuró un pipeline de Integración Continua y Despliegue Continuo (CI/CD) mediante GitHub Actions, utilizando un runner autoalojado (self-hosted) que ejecuta directamente sobre la máquina virtual del laboratorio.

Justificación del uso de un runner autoalojado

Un runner alojado en la infraestructura de GitHub (cloud) no tiene visibilidad de la red privada 172.22.x.x del laboratorio ni del clúster de Minikube ejecutándose en la máquina física. Por esta razón, el pipeline requiere obligatoriamente un runner self-hosted, instalado como servicio en la misma máquina virtual donde corre el ecosistema completo.

Etapas del pipeline

El flujo automatizado ejecuta, en orden estricto, las siguientes operaciones: construcción de la imagen Docker dentro del daemon de Minikube, creación idempotente del namespace, inyección de los Secrets desde las credenciales seguras de GitHub (Settings → Secrets and variables → Actions, nunca en texto plano dentro del repositorio), aplicación de los manifiestos de PersistentVolumeClaims, ConfigMaps, Deployments y Services, espera activa hasta que los cuatro Deployments alcancen el estado Ready, y finalmente la aplicación de las NetworkPolicies de Zero Trust — deliberadamente como último paso, para no bloquear el tráfico de arranque inicial de los propios contenedores.

Etapas del pipeline (12 pasos)

Paso	Nombre	Descripción
1	Checkout código	Descarga del código del repositorio
2	Verificar clúster	kubectl cluster-info — falla si el clúster no está accesible
3	Construir imagen	eval \$(minikube docker-env) && docker build -t inventario-app:latest .
4	Crear namespace	kubectl apply -f k8s/namespace/namespace.yaml (idempotente)
5	Crear Secrets	Crea/actualiza los 4 secrets desde GitHub Secrets (--dry-run=client kubectl apply)
6	Aplicar PVCs	kubectl apply -f k8s/pvc/
7	Aplicar ConfigMaps	kubectl apply -f k8s/configmaps/

8	Aplicar Deployments + Services	kubectl apply -f k8s/deployments/ && k8s/services/
9	Esperar rollout	kubectl rollout status para los 4 deployments (timeout 120s)
10	Aplicar NetworkPolicies	kubectl apply -f k8s/networkpolicies/ — deliberadamente al final, para no bloquear el arranque inicial
11	Estado final del clúster	kubectl get pods, services, networkpolicies + URL de la app
12	Probar /api/health	curl /api/health (continue-on-error: true — informativo)

Evidencia de ejecuciones exitosa

La siguiente captura corresponde a la ejecución de un comando en sql-server en lenguaje sql para poder visualizar los permisos del usuario “docente01”

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'inventario_itu' database schema with various tables and views. The main window shows a query editor with the following SQL command:

```
show grants for 'docente01'@'%';
```

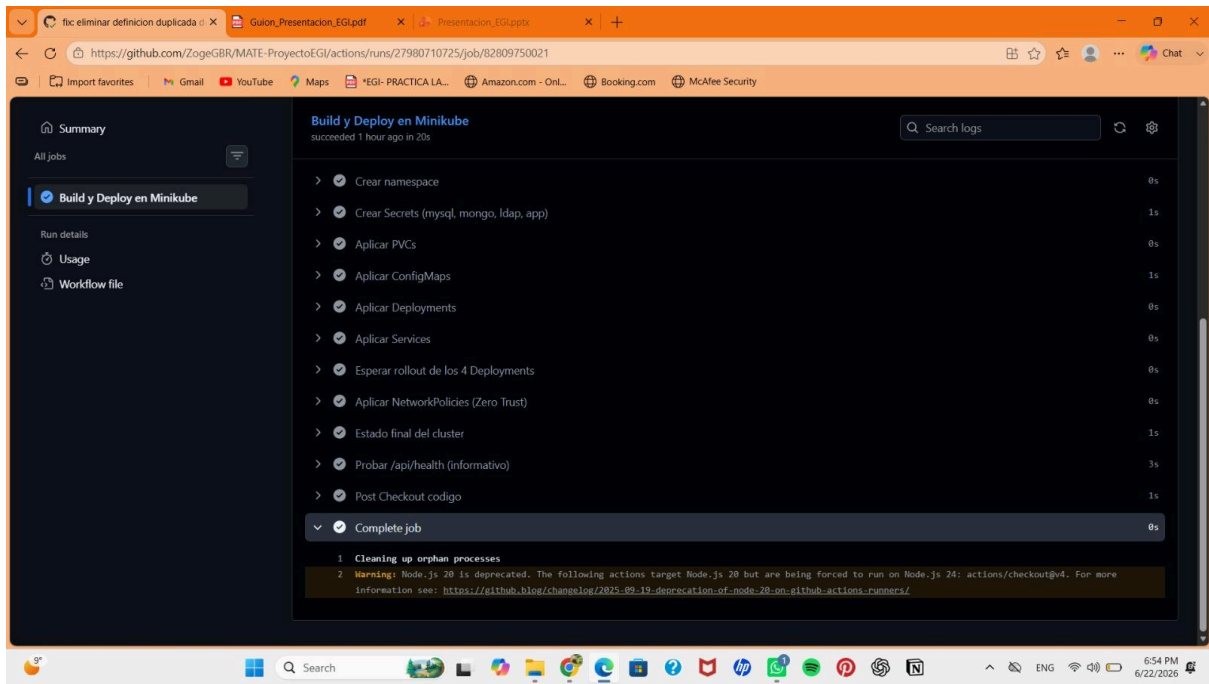
The query results are displayed in the 'Result Grid' tab, showing the following output:

Grants for docente01@%
GRANT USAGE ON *.* TO 'docente01'@'%'
GRANT SELECT ON 'inventario_itu'.* TO 'doc...

Below the query results, the 'Action Output' tab shows a log of database actions and their messages:

#	Time	Action	Message	Duration / Fetch
28	19:19:38	show grants for 'docente01'@'%'	2 row(s) returned	0.000 sec / 0.000 sec
29	19:21:36	drop table equipos	Error Code: 3730. Cannot drop table 'equipos' referenc...	0.000 sec
30	19:21:58	show grants for 'docente01'@'%'	2 row(s) returned	0.000 sec / 0.000 sec
31	19:34:21	show grants for 'docente01'@'itu.local'	Error Code: 1141. There is no such grant defined for us...	0.000 sec
32	19:34:29	drop table equipos	Error Code: 3730. Cannot drop table 'equipos' referenc...	0.000 sec
33	19:35:13	show grants for 'docente01'@'%'	2 row(s) returned	0.000 sec / 0.000 sec
34	19:35:36	drop table equipos	Error Code: 3730. Cannot drop table 'equipos' referenc...	0.000 sec
35	19:36:30	show grants for 'docente01'@'%'	2 row(s) returned	0.000 sec / 0.000 sec

La siguiente captura corresponde a la ejecución más reciente del workflow 'Build y Deploy en Minikube', con resultado succeeded en sus 12 pasos:



Componentes de infraestructura

Verificación	Estado
Pipeline de CI/CD ejecutado exitosamente (12/12 pasos)	OK
Los 4 Deployments en estado Ready sin duplicados	OK
Conectividad de red verificada hacia pfSense (puertos 389 y 3306)	OK
GUPW activo con las 5 reglas mínimas requeridas	OK

Funcionalidad de la aplicación

Verificación	Estado
Endpoint /api/health reporta status: ok (sql, mongo y ldap conectados)	OK
Login funcional contra Active Directory real (formato userPrincipalName)	OK
Consulta de inventario contra SQL Server real de Windows	OK
Vinculación correcta entre MySQL y MongoDB mediante mongo_id	OK

Cumplimiento de la consigna

Requisito de la consigna	Estado
Conectividad SQL/MongoDB y vinculación por ID	OK
Autenticación contra Active Directory/LDAP institucional	OK
Despliegue contenerizado mediante Kubernetes	OK
Zero Trust con NetworkPolicies y CNI compatible (Calico)	OK
Firewall perimetral simulado (pfSense + GFW)	OK
Repositorio Git versionado con historial de evolución	OK

El sistema cumple, en su estado actual, con la totalidad de los requisitos técnicos y de seguridad exigidos por la consigna del Proyecto Integrador, validados mediante un proceso de testing documentado y un pipeline de integración continua reproducible.